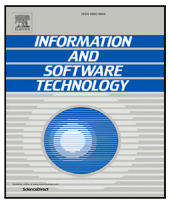
Information and Software Technology 189 (2026) 107929

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



Review article

Automatic test case generation using natural language processing: A systematic mapping study

Jordy Navarro, Ronald Ibarra *

Grupo de Investigación en Ingeniería de Software, Universidad Nacional Agraria de la Selva, Tingo María, Peru

ARTICLE INFO

Keywords:
Test case generation
Natural language processing
Test case design
Artificial intelligence
Nlp

ABSTRACT

Context: Artificial intelligence (AI) has made significant progress in recent years, which has motivated its use in many disciplines and industrial domains, including software engineering, especially in the testing process, where many research efforts have been made. These studies focus on the automatic test case generation using natural language processing (NLP), an emerging branch of AI. Despite these efforts, the literature lacks a structured and systematic approach, since reported mappings and systematic literature reviews have limitations in their scope.

Objective: This study aims to systematically organize and synthesize the existing literature to establish the state of the art in the automatic generation of test cases using NLP.

Methodology: We conducted systematic mapping following Kai Petersen's methodology, exploring five databases. The initial search yielded 1262 articles, of which 61 were selected. 16 thematic questions and 4 non-thematic questions were posed.

Results: The findings reveal an increase in the number of articles published in journals starting in 2022. Among the most reported NLP techniques are POS tagging, dependency parsing and tokenization, implemented with tools such as Stanford Core NLP and NLTK. The reported approaches mostly achieved a medium level of automation, using natural and formal language requirements as main inputs. Only 9 articles explicitly mention the use of test case design techniques, such as boundary value analysis, equivalent class partitioning, state transition and decision tables.

Conclusions: We systematically identified and organized the reported primary studies on the automatic or semi-automatic generation of software test cases applying NLP.

Contents

* Corresponding author.
E-mail addresses: jordy.navarro@unas.edu.pe (J. Navarro), ronald.ibarra@unas.edu.pe (R. Ibarra).

https://doi.org/10.1016/j.infsof.2025.107929
Received 24 June 2025; Received in revised form 9 September 2025; Accepted 12 October 2025
Available online 22 October 2025

1. Introduction

Software testing is intended to evaluate an attribute or capability of a program to verify whether it meets the required specification [1]. Today, software testing has become a highly automated process; however, for this process to be carried out correctly, it must be properly organized and managed. This task is challenging due to the complexity of the activities involved [2].

One of the main activities in the software testing process is test case generation [3], also known as test case design [4,5]. Traditionally, test engineers perform this activity manually, a process that can be labor intensive, costly, and error-prone, especially when test cases are embedded within large volumes of information with varying levels of complexity [6]. However, with advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP), new methodologies and tools have emerged to automate test case generation from various data sources. These methodologies and tools represent a valuable approach to improving the productivity and reliability of the testing process, which plays a crucial role in delivering high-quality software products [7].

Large Language Models (LLMs) have revolutionized the field of AI, demonstrating remarkable performance in NLP tasks [8]. However, these models operate as 'black boxes', making it difficult to understand the rationale behind their decisions; such lack of transparency may reduce the trust and acceptance of their results in the software industry [9]. In contrast, approaches based solely on NLP provide greater visibility into the procedures and techniques used. At the same time, there are hybrid studies that combine NLP techniques (e.g., for preprocessing) with LLM (e.g., for code generation) to achieve more effective outcomes.

This study presents state-of-the-art research applying NLP techniques for software test case generation, without excluding those that incorporate LLMs at some stage of the process. This approach not only helps to understand the evolution of the topic, but also identifies knowledge gaps that create research opportunities. The systematic mapping methodology used in this study follows the guidelines proposed by Kai Petersen [10], which provide a rigorous and reproducible approach to structure and organize the search and selection of studies.

This article is structured as follows: Section 2 presents related works, Section 3 describes the methodology used, Section 4 presents the obtained results, Section 5 presents discussions of the results, Section 6 analyzes threats to validity and Section 7 provides the conclusions derived from the literature review.

2. Related work

When reviewing the literature, secondary studies have been found that focus on organizing reported information about the application of artificial intelligence and NLP in the software testing process, as well as more specific secondary studies directly related to test case generation. However, the latter have a limited scope in their research questions, which has motivated the development of this study.

2.1. The role of AI in the testing process

In [11,12], the role of AI in the software testing process is studied by analyzing the different AI techniques and software testing activities to which they can be applied. Test case generation is the activity most frequently addressed in the proposed approaches. By applying techniques such as machine learning, artificial neural networks (ANN), NLP, among others, the studies concluded that integrating AI techniques into the software testing process simplifies and enhances software testing activities in terms of test case reuse, reduction of manual effort, improved coverage and fault detection.

In another review [13], studies on software testing that utilize machine learning methods were identified. The review covered studies from 2018 to 2024 and identified four main methods: supervised learning, unsupervised learning, reinforcement learning, and hybrid learning methods, with the first and the last accounting for 74% of the articles. Python emerged as the predominant tool and platform in 39% of the reviewed articles. Although this study provides information on the methods and tools used, it does not mention relevant details about their applicability in the software testing process.

Other studies have reported research focused on LLM-based software testing, highlighting the strong capability of these models to generate test inputs [8] and unit test cases [9,14,15]. However, several limitations have also been identified, including: the restricted scope of the generated test types [8,15]; the low-test coverage achieved [8];

Table 1
Comparison of related works.

Ref.	Scope	Time-Span	# of papers	Sources	# of RQs	RQs keywords
[6]	NLP-based test case generation	2005–2016	Initial: 3195 Final: 16	ACM, IEEE, SPRINGER	2	techniques, tools
[7]	NLP-based test case generation	2015–2023	Initial: 416 Final: 13	ACM, IEEE, SPRINGER	3	tasks, techniques, tools
[16]	NLP-based software testing	2001–2017	Initial: 95 Final: 67	Scopus, Google Scholar	11	approach, algorithms, inputs, models, tools, languages, test artifacts, questions, evaluation method, precision
[17]	NLP-based software testing	2013–2023	Initial: 453 Final: 24	Scopus, Web of Science	10	techniques, algorithms, language, test types, ML and DL algorithms, tools, approach reliability, dataset, challenges, domain
This work	NLP-based test case generation	2004–2024	Initial: 1262 Final: 61	Scopus, IEEE, Web of Science, ACM, Science Direct	16	techniques, tools, models, test design technique, inputs, test case format, test case information, test types, quality characteristic, performance, coverage level, automation level, domain, contribution type, validation type, project type

the dependency on diverse, high-quality training data [9]; and the lack of validation of the proposed approaches in real software or systems [14]. These limitations hinder their implementation in real industrial scenarios.

Additionally, a mapping study [16] and a review [17] focusing on the use of NLP in the testing process were found. Finally, two reviews [6,7] were identified that focus on the use of NLP in a more specific part of the testing process, namely in test case generation.

2.2. NLP in test case generation

In [16] they focused on performing systematic mapping that covered the latest developments in NLP-assisted software testing, analyzing a total of 67 articles from 2001 to 2017 and identifying a total of 38 NLP-based tools applicable to the testing process. The results showed that only 11% of the tools found were available for free download and use, and a large proportion of the articles (30 out of 67) provided only superficial treatment of NLP aspects.

Additionally, [17] presents a systematic review aimed at providing an in-depth analysis of the current body of literature on the emerging topic of NLP-based software testing. A search for relevant articles was carried out in the Scopus and Web of Science databases between 2013 and 2023, yielding a total of 453 articles, of which 24 were selected. After the review, the author concluded that the advances in integrating NLP techniques in various testing scenarios are quite promising, especially in test case generation and requirements analysis, which are the current trends.

Both [16,17] adopt a general approach that covers most of the software testing activities defined in SWEBOK [3], such as test planning, test generation (which is the focus of this study), and test report, without exploring the specific details of test case generation, which is the main objective of this study.

In [6,7] the role of NLP in automatic test case generation was explored. The review in [6] covered the years 2005 to 2016, resulting in a final set of 16 articles, within which the following were identified: 6 NLP techniques, 18 tools, 9 algorithms, and 4 approaches. On the other hand, in [7] 13 articles were selected between 2015 and 2023, identifying the following: 3 NLP techniques, 2 tools, 1 framework, and 7 NLP algorithms. Furthermore, the authors agree that NLP-based tools and methods constitute a valuable approach to enhancing the effectiveness, reliability, and efficiency of software test case generation.

Studies [6,7] represent direct competitors of this work, as they pursue a similar scope. However, both reviews include only three research questions, which are insufficient to ensure a comprehensive view of the topic. Table 1 presents a comparison of the secondary studies that are most closely aligned with the purpose of our work.

3. Methodology

To conduct this study, we followed the guide for developing systematic mapping studies in software engineering proposed by Petersen [10].

3.1. Research questions

After reviewing the secondary studies on the application of NLP in automatic test case generation, only two studies (i.e., [6,7]) that specifically addressed this aspect were identified. However, as both are systematic reviews, their research questions lack the level of comprehensiveness required to establish the state-of-the-art review presented in this work, which encompasses 16 thematic (topic-specific) questions and 4 non-thematic (topic-independent) questions, as summarized in Tables 2 and 3, respectively.

3.2. Search strategy

3.2.1. Data sources

Five digital databases were chosen: Scopus, Science Direct, IEEE Xplore, ACM Digital Library, and Web of Science, due to their scientific relevance and popularity in the field of software engineering.

3.2.2. Search string

The PICO method was used to define the search string; however, only PI was used (see Table 4), considering the term “test case” as the population (P) and the terms “generation” and “natural language processing” as the intervention (I). In addition, alternative terms were selected based on a keyword analysis of a set of 10 previously known reference articles. This method was used to develop the following base search string:

Base String: (“test case” OR “test specifications” OR “test scenarios” OR “test suite” OR “test scripts” OR “test plan”) AND (“generation” OR “generating” OR “generate” OR “creation” OR “design”) AND (“natural language processing” OR “nlp” OR “natural language” OR “computational linguistics”)

3.2.3. Search evaluation

To validate the search string, ten known articles were used, which also served to identify the alternative terms included in the string. A pilot search was then conducted to verify that these articles were present in the set of results retrieved from the databases selected for this study, and their inclusion was confirmed. In addition, information was extracted from these articles to validate the research questions formulated in Section 3.1.

Table 2
Topic-specific questions and their motivations.

Id	Research questions	Motivation
RQ.1	What NLP techniques are used to generate test cases?	Knowing which techniques, tools and algorithms are used will help guide future empirical work.
RQ.2	What are the main NLP tools that have been used or proposed for generating test cases?	
RQ.3	What algorithms/models have been used to generate test cases?	
RQ.4	What test case design techniques were applied?	Knowing the test case design techniques allows you to understand the completeness of the scope of the proposed approaches.
RQ.5	What was the input for the methods used for test case generation?	Understand the complexity of input data for automated test case generation for the purpose of replicating or extending existing work.
RQ.6	In what format are the test cases generated in the proposed solutions?	Understand the completeness of automatically generated test cases to identify gaps in the usefulness of their implementation in the industry.
RQ.7	What information is present in the generated test case?	
RQ.8	To which types of tests are the generated test cases oriented?	
RQ.9	What quality characteristics or sub-characteristics have been tested?	
RQ.10	What performance is reported when applying the proposed solutions?	Provide references for future research through replication, extension or new proposals.
RQ.11	What level of test coverage was achieved?	
RQ.12	What level of automation was achieved?	
RQ.13	What is the type and domain of the software used to validate the proposal?	Evaluate the feasibility of its implementation in the industry.
RQ.14	What was the type of contribution of the proposed solutions?	Assess the scope and diversity of the proposed solutions.
RQ.15	How were the proposed solutions validated?	
RQ.16	How many studies were applied to real projects?	

Table 3
Topic-independent questions and their motivations.

Id	Bibliometric questions	Motivation
BQ.1	What types of events and in what year were the research studies on test case generation using NLP published?	Understand the evolution of the topic over time and the authors' preferences when publishing research in this specific area.
BQ.2	Who are the main authors working on studies for generating test cases using NLP?	Establish collaborative networks (work as a team).
BQ.3	What is the volume of research on the topic by geographic region?	
BQ.4	Which works are the most influential on the topic?	Identify studies that can serve as background for future primary research, ensuring they include a relevant theoretical framework.

Table 4
PI method for search string.

PI	Main term	Alternate terms
Population: test case	test case	test specifications, test scenarios, test suite, test scripts, test plan.
Intervention: generation using natural language processing	generation	generating, generate, creation, design.
	natural language processing	nlp, natural language, computational linguistics.

3.3. Study selection

For this process, the JabRef tool was used, which enabled the screening of titles, keywords, and abstracts to classify the articles into the groups 'included' and 'excluded'. The initial selection was carried out by the first author, while the second author reviewed each phase of the process without finding any discrepancies.

To ensure objectivity in the primary study selection process, inclusion and exclusion criteria were defined to minimize biases. These criteria help to delimit the scope of the mapping study by guiding

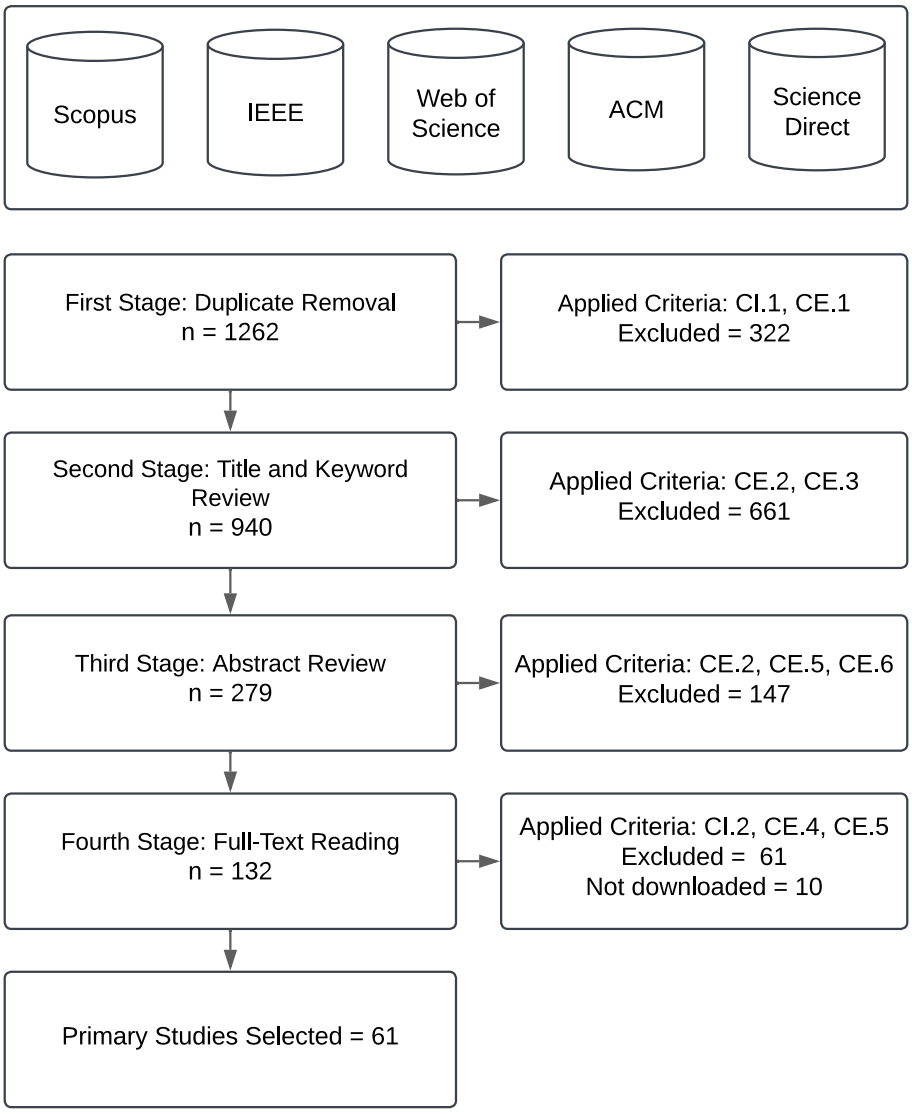


Fig. 1. Search and selection process.

the inclusion of relevant studies and excluding those that are not. The following inclusion and exclusion criteria were applied during a process consisting of 4 stages (see Fig. 1).

Inclusion Criteria

- CI.1 Studies in which a technique, tool, or approach for generating test cases using NLP has been applied were included.
- CI.2 Primary and empirical studies that provide results or evidence of having applied a technique, tool, or approach for generating test cases using NLP will be accepted.

Exclusion Criteria

- CE.1 Duplicate studies.
- CE.2 Articles whose title or keywords are not related to the generation of test cases with NLP were excluded.
- CE.3 Secondary and tertiary studies, books, and conference abstract proceedings were excluded.
- CE.4 Articles not written in English were excluded.
- CE.5 Successive articles were excluded, accepting only the most recent.
- CE.6 Articles that, despite having relevant terms in the title and keywords, did not focus on generating software test cases with NLP were excluded.

3.4. Data extraction

For this process, a shared spreadsheet was created by both authors to record the information extracted from each primary study, with the purpose of categorizing them through an iterative process, as suggested by Petersen. Data extraction was carried out by the first author, and to ensure quality, the second author conducted a review in which some discrepancies were identified and subsequently resolved by consensus.

4. Results

4.1. Summary of studies

After applying the proposed review methodology, 61 articles were selected. Of these, 26% were published in journals and 74% at conferences. Fig. 2 illustrates the increasing trend of studies published in both journals and conferences, suggesting that the topic is actively developing within the scientific community.

To assess the geographical distribution of the studies, the country of affiliation of the first author was considered. Fig. 3 shows that the most productive countries in this research area include the United States, India, Germany, China, Brazil, and Luxembourg, each contributing between 5 and 9 publications. Some researchers participate in multiple

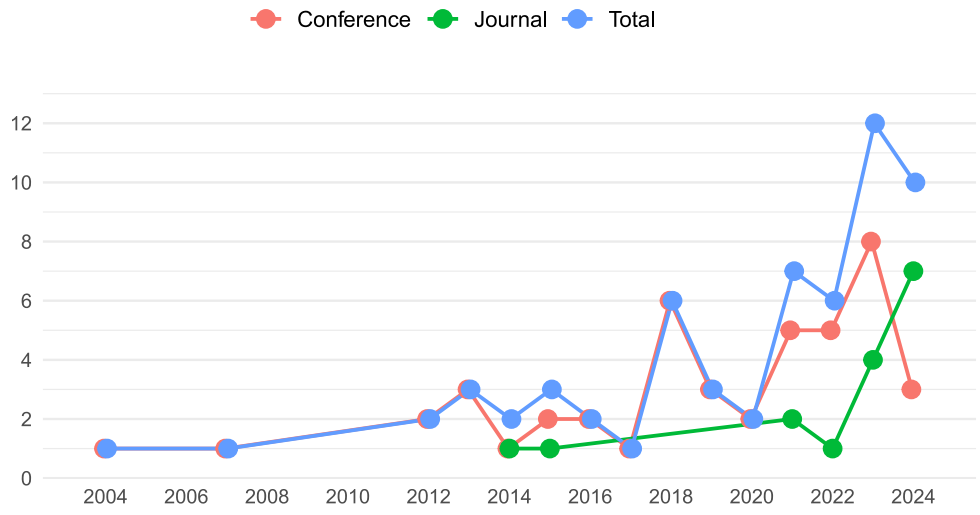


Fig. 2. Articles published in journals and conferences per year.

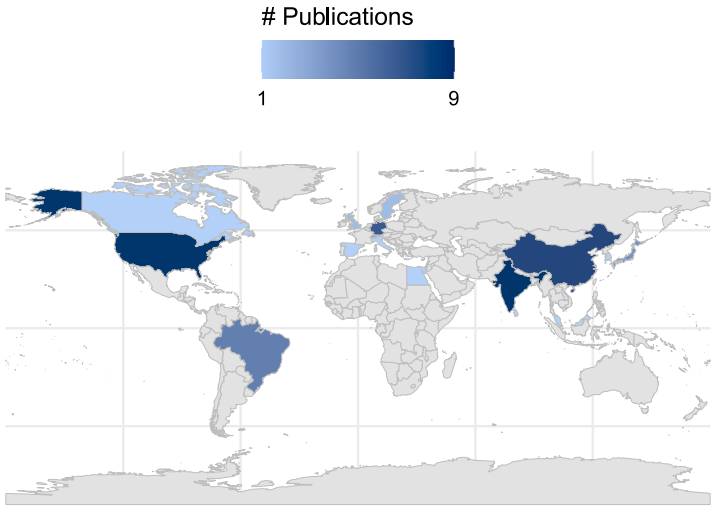


Fig. 3. Articles published by country.

Table 5
Most influential authors.

# of Contribution	Authors
5	Briand Lionel, Pastore Fabrizio
4	Barros Flávia, Carvalho Gustavo, Goknil Arda, Wang Chunhui, Fischbach, Jannik
3	Ernst Michael D, Gorla Alessandra, Pezzè Mauro, Vogelsang Andreas
2	A total of 22 authors
1	A total of 184 authors

Table 6
Most cited articles.

Ref.	# Citations	Country	Year
[18]	90	Luxembourg	2015
[19]	82	Switzerland	2016
[20]	73	Germany	2019
[21]	64	Switzerland	2018
[22]	54	USA	2018
[23]	47	South Korea	2023
[24]	38	Brazil	2014
[25]	31	Germany	2020
[26]	25	Luxembourg	2022
[27]	23	Brazil	2013

contributions (see Table 5) as both authors and co-authors, and some even collaborate on the same research projects, indicating the presence of at least one active research group in this field in recent years. Additionally, the most cited-and thus arguably the most influential-articles in this area were identified. Table 6 presents the 10 most cited articles.

4.2. RQ.1: What NLP techniques are used to generate test cases?

In the reviewed articles, a wide variety of NLP techniques have been found to automate the test case generation process (see Fig. 4). The most used technique in these articles is POS (Part of Speech) tagging, which involves assigning grammatical labels (noun, verb, adjective) to each word in a sentence according to its context and function [26]. This is followed by dependency parsing, which analyzes the grammatical structure of a sentence to determine the relationships between words [28]. Other techniques used include tokenization, which splits text into smaller units (words, phrases, or sub words) [29], and named

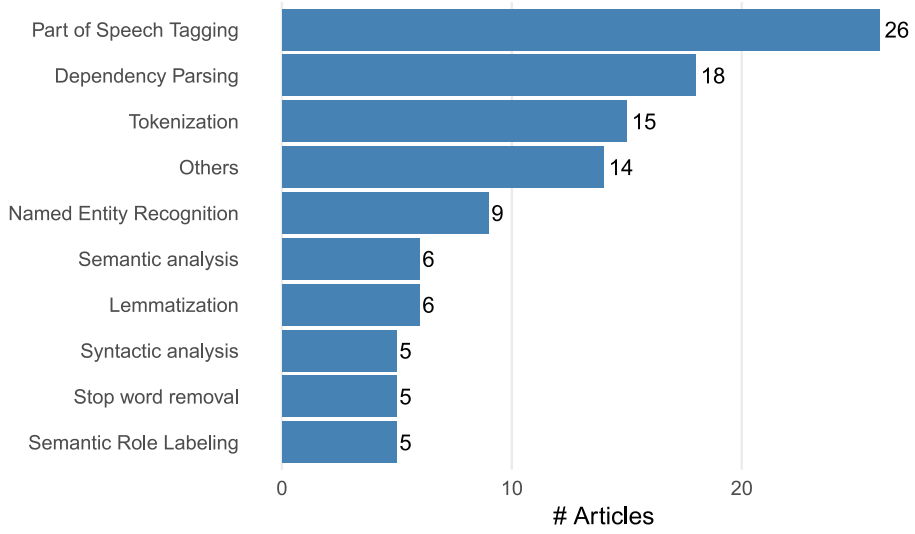


Fig. 4. Most used NLP techniques.

Table 7
NLP tools identified.

NLP Tools	#	Articles
Stanford Core NLP	10	[20,22,31–38]
NLTK	6	[35,39–43]
spaCy	4	[39,42,44,45]
Stanford Parser	3	[19,21,46]
SysReq-CNL, NAT2TEST	3	[24,47,48]
GATE	3	[18,49,50]
CNL-Parser	3	[24,27,47]
WordNet	2	[43,51]
Others	15	NLCI [52], Textacy [44], Toradocu [19], SentencePiece [53], Stanza [51], Pyspellchecker [42], RestTestGen [41], RETNA [54], Link Grammar parser [55], NLReq2TC [27], ChatGPT [56], Jieba [57], KeyBERT [35]

entity recognition (NER), which identifies and classifies entities (names, locations, dates, organizations) based on predefined categories [30].

4.3. RQ.2: What are the main NLP tools that have been used or proposed to generating test cases?

The tools and libraries used in the selected studies are listed in Table 7. Among the most frequently used NLP tools are Stanford Core NLP, which supports tasks such as tokenization, POS tagging, dependency parsing, NER, and sentiment analysis. NLTK is a Python library for tokenization, POS tagging, stemming, lemmatization, and text analysis. Another tool identified is spaCy, a modern and efficient Python library that includes functionalities such as tokenization, POS tagging, NER and dependency parsing.

4.4. RQ.3: What algorithms/models have been used to generate test cases?

With the advancement of NLP, large language models (LLMs) have become increasingly relevant in various software engineering applications, including unit test case generation, language translation, code summarization, among others, since LLMs can understand and generate human-like text [56]. These models internally utilize NLP techniques and processes to achieve their purpose. The review revealed that several studies employ LLMs to perform specific tasks and procedures, in combination with classical NLP techniques, to generate test cases. Among the models reported most frequently in primary studies is GPT-3.5 Turbo, developed by OpenAI and based on the Transformer architecture. Another frequently mentioned model is BERT, which processes word context bidirectionally and was developed by Google. Table 8 presents additional models and algorithms used in the studies to support their approaches.

4.5. RQ.4: What test case design techniques were applied?

Test case design techniques are activities that enhance coverage and ensure that the tests are specific to the situation being evaluated [68]. However, the primary studies in this mapping mostly do not report on the use of test case design techniques. As shown in Table 9, only nine articles implemented some of these techniques in their proposals.

4.6. RQ.5: What was the input for the methods used for test case generation?

To generate test cases, the authors propose solutions that require an artifact input. This artifact must contain the necessary information from which the test cases can be derived. The artifacts used are shown in Fig. 5, with the most utilized ones being software requirements (in natural and formal language), user stories and use cases. In particular, prompts are mainly employed in methods based on large language models (LLMs), where they can combine multiple sources of information, such as requirements, bug reports, or code fragments. Finally, the 'Others' category includes less frequent inputs, such as application forms, project paths, comments, and screenshots.

4.7. RQ.6: In what format are the test cases generated in the proposed solutions?

When referring to the test case format, we mainly distinguish between two categories:

Natural Language Specification (NLS): This is when the test case is described in natural language, detailing the steps and test data.

Test Script (TS): This is an executable test case written in a programming language.

The difference is that executable test cases go a step further than natural language specifications, as the latter require manual translation into executable code.

Table 8
Algorithms/Models Identified.

Algorithm/Model	#	Articles
GPT 3.5 Turbo	5	[56,58–61]
BERT	3	[62–64]
RoBERTa	2	[63,65]
T5	2	[61,66]
Bi-LSTM-CRF	2	[39,57]
Word Mover's Distance (WMD) algorithm	1	[21]
SentenceBERT	1	[62]
Word2Vec	1	[67]
CAT-LM	1	[53]
Codex	1	[23]
GPT-4, Llama-2-13B, PaLM-2	1	[60]
en_core_web_trf	1	[44]

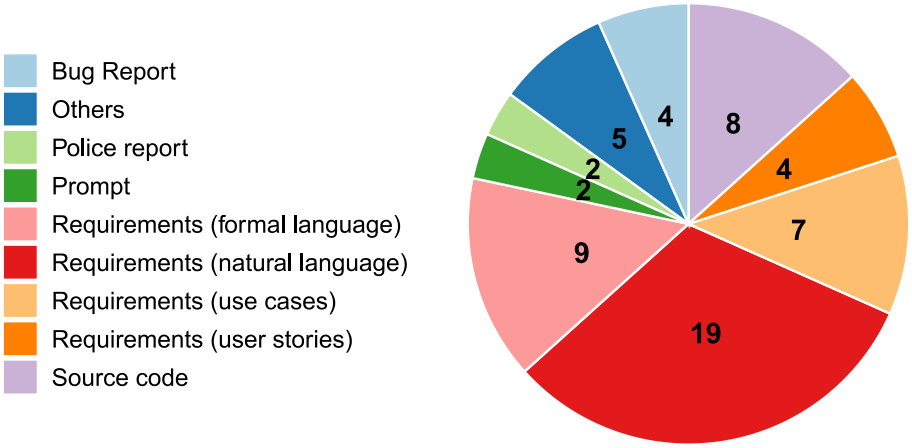


Fig. 5. Method inputs.

Table 9
Applied test case design techniques.

Technique	#	Articles
Boundary Value Analysis	4	[36,37,69,70]
Equivalence Partitioning	3	[36,37,69]
State Transition	3	[45,47,54]
Decision Table	2	[38,71]

Fig. 7 presents the distribution of these two formats, indicating a higher frequency of test cases generated in the natural language specification format, which were mostly evaluated using requirements coverage. In contrast, test scripts were evaluated with code coverage metrics (e.g., paths, branches, lines, statements, and methods), which are discussed in more detail later.

4.8. RQ.7: What information is present in the generated test case?

When designing a test case, many aspects must be considered, including the information necessary to properly execute the test. At its most basic, a test case contains, at minimum, the input data and the expected result [72]. In other cases, additional complementary information (such as test steps, preconditions and postconditions, among others) may be required. Fig. 6 shows how the information contained in the test cases is distributed. Among the most frequently reported elements are test inputs, preconditions (necessary prior conditions for the test), and test steps, which outline the workflow to follow when executing the test.

4.9. RQ.8: To which types of tests are the generated test cases oriented?

Test types represent a categorization or classification based on the objectives they must meet. In general, the generated test cases are functional tests, aimed at verifying the specific functionality of the software. There are various types and/or categories of tests, and each author classifies them differently (see Fig. 7). Among those identified in this study are:

Functional Tests (FT): Verify that the system meets the specified functional requirements.

Unit Tests (UT): Evaluate individual components of the code, such as functions or methods, in isolation.

User Interface Tests (UIT): Validate that the graphical interface works correctly and is intuitive for the user.

System Tests (ST): Evaluate the complete system to ensure that all parts work together as expected.

Regression Tests (RT): Ensure that changes in the system have not introduced errors in already existing functionalities.

Acceptance Tests (AT): Confirm that the system meets the customer's requirements and is ready for use.

4.10. RQ.9: What quality characteristics or sub-characteristics have been tested?

The ISO 25010 standard (which defines quality models for software) was used as a reference to determine the quality characteristic intended

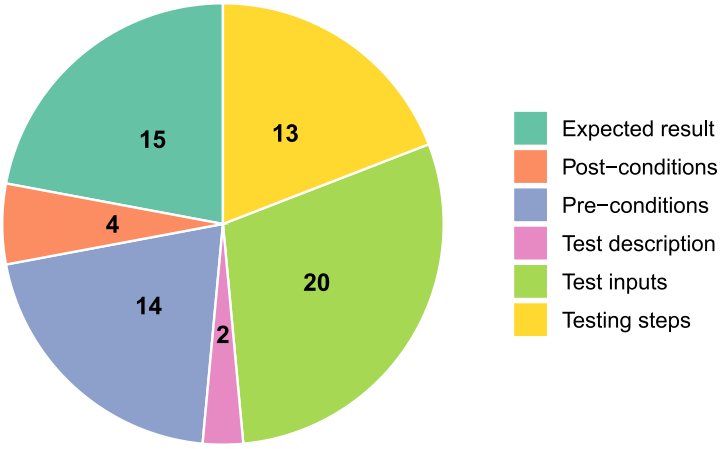


Fig. 6. Information presented in the test cases.

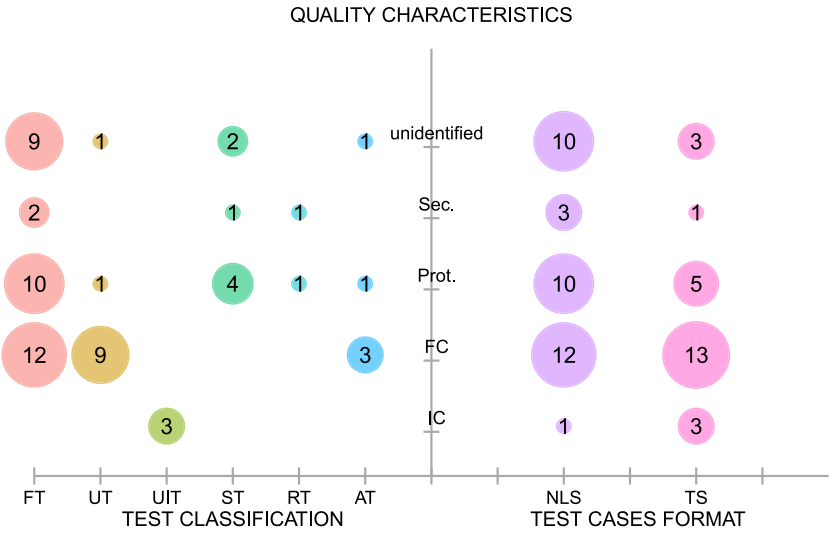


Fig. 7. Test classification, quality characteristic and test case format.

to be evaluated in each study. In several cases, it is easy to deduce which quality characteristic is being assessed based on the type of test and the software used. Fig. 7 illustrates the quality characteristics that were deduced, including:

Interaction Capability (IC): The ability of the software to allow users to interact with its interface by exchanging information to complete specific tasks.

Functional Correctness (FC): Measures the accuracy and completeness with which the system meets the expected results according to the functional requirements.

Protection (Prot): The ability of the product, under defined conditions, to prevent a state that could endanger human life, health, property, or the environment.

Security (Sec): Involves the protection of information and data to ensure confidentiality, integrity, and availability.

The most frequently mentioned quality sub-characteristic of the ISO 25010 model is functional correctness, which is assessed in most studies using functional and unit testing. There was also an even distribution in terms of the test format used (natural language specifications and test scripts). Conversely, Security and Interaction Capability were the least frequently addressed quality characteristics in the reviewed studies, the latter being closely related to user interface testing.

4.11. RQ.10: What performance is reported when applying the proposed solutions?

The metrics associated with the approaches presented in the selected articles are reported in Fig. 8. These metrics, which include accuracy (the overall correctness), F1 score (the harmonic mean of precision and recall), precision (the proportion of true positive results), and recall (the ability to identify all relevant cases) serve as key indicators of the approaches, tools and algorithms performance.

Of the total 61 studies, 40 reported using at least one metric, with 31 of them specifically reporting the use of precision.

Regarding performance, 22 articles reported a precision between 80% and 100%, while only 4 articles [22,25,41,59] reported a precision below 60%, indicating that the approaches, algorithms, and tools proposed in this field demonstrate acceptable performance.

4.12. RQ.11: What level of test coverage was achieved?

Test coverage measures the extent to which a system has been verified by identifying uncovered areas and detecting when the testing process becomes ineffective. In addition, it helps to generate new test cases to improve verification and optimize regression suites [73].

In the test coverage results presented in Fig. 9, it was observed that all studies reporting requirements coverage and line coverage

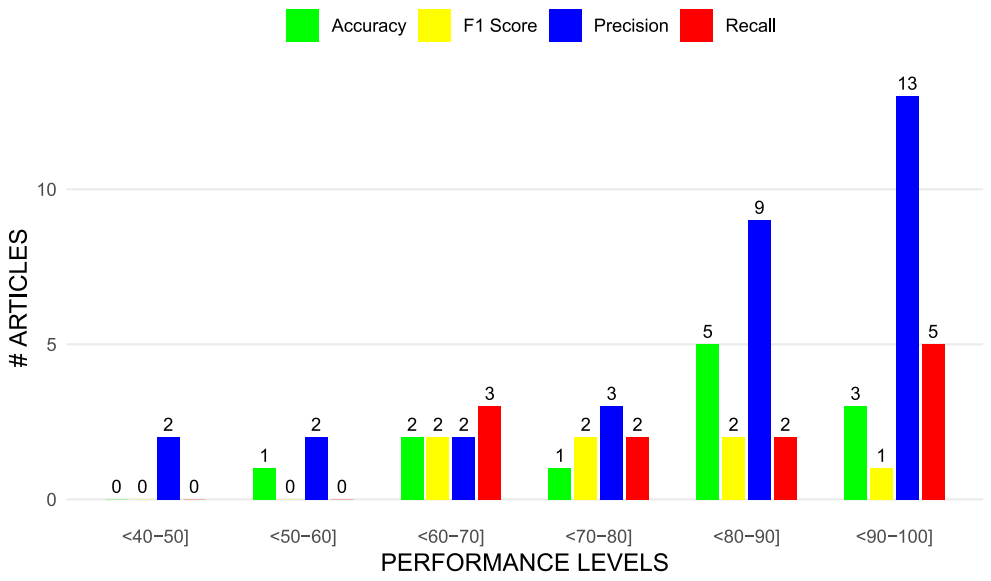


Fig. 8. Accuracy, F1-Score, Precision and Recall.

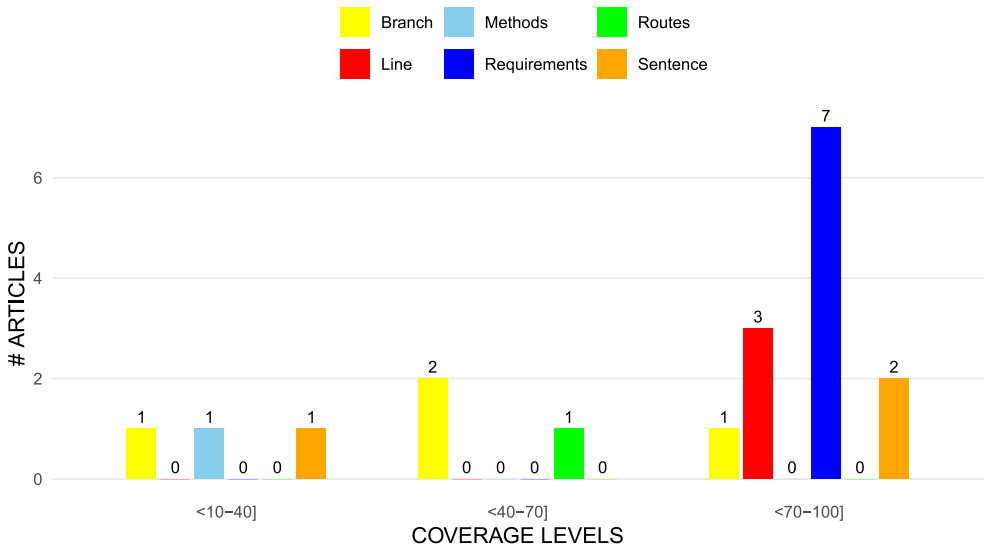


Fig. 9. Test coverage levels.

achieved high levels (70%–100%). In contrast, among the four studies that reported branch coverage, three reached lows to medium levels, suggesting the inherent complexity of applying this type of coverage. Similarly, only one study reported method coverage and another reported path coverage, with low and medium levels, respectively. Finally, three studies reported statement coverage, of which two achieved high levels and one a low level. This analysis suggests that the reviewed literature prioritizes requirements and line coverage, while aspects such as path and branch coverage generally receive less attention.

This study also identified other metrics used to evaluate the generated test cases, such as fault detection rate [21,23,33,52,74,75], mutation score [24,27,37,47,48,74], and compilability rate [53,58].

Finally, the tools used in the studies to measure coverage, and other reported metrics were identified; these tools are presented in Table 10.

4.13. RQ.12: What level of automation was achieved?

The classification of the automation level of the approaches was based on the number of testing process activities defined in SWEBOOK [3] that were reported as automated. To this end, the following classification criteria were developed:

High: Approaches that cover multiple process activities (requirements analysis, test generation, and execution) and include advanced activities such as script generation, simulations, or automatic validation.

Medium: Approaches that automate at least two process activities.

Low: Approaches that automate only a single activity within one phase or are limited to automatically performing basic tasks such as information extraction or classification.

The classification is presented in Fig. 10, which shows that most of the approaches have a medium level of automation. The data suggests that there are still gaps to be addressed in the automation of the testing process.

4.14. RQ.13: What is the type and domain of the software used to validate the proposal?

When generating test cases, it is important to know the type of software under evaluation, as it often influences test case complexity. This study identified the following software types:

Table 10
Tools for metric evaluation.

Tool	Language	Metric type	Detail	Ref.
JaCoCo	Java	Line, branch, instruction and method coverage	Open source	[41,53]
GCOV	C, C++	Line, branch and method coverage	Oper source	[58]
Nyc	JavaScript, TypeScript	Line, sentence, method and branch coverage	Open source	[59]
EclEmma	Java	Line, branch, instruction and method coverage	Open source	[76]
Coverage	Python	Sentence and branch coverage	Open source	[53]
Major	Java	Mutation testing	Open source	[37]
μ Java	Java	Mutation testing	Open source	[47]

Web App (W. App): Applications accessible through web browsers, designed to interact with end users and provide dynamic functionalities.

Web Api (W. Api): Programming interfaces that allow different applications to communicate via HTTP protocols, generally for data exchange.

Web Service (W. Serv): Web-based services that enable interoperability between systems using standard protocols like SOAP or REST, typically used to integrate applications.

Mobile: Applications designed for mobile devices such as phones and tablets, optimized for small screens and mobile operating systems (iOS, Android).

Embedded (Emb): Software developed for embedded systems—hardware devices with dedicated software for specific tasks.

Class Library (Class.Lib): A set of reusable code components organized in classes, used to expedite application development by providing specific functionalities.

Additionally, the software domain is essential when designing test cases. Evaluating a system from the automotive or industrial domain, where physical user protection is prioritized, differs from assessing software in the financial or business domain, which prioritizes information and data security. According to Fig. 11, for embedded software, the most frequent domains are automotive and industrial; for mobile systems, Web services, and Web APIs, the entertainment domain stands out; and for Web Apps, the business, entertainment, and IT domains are highlighted. Other domains identified include academic, transportation, and air transportation software.

4.15. RQ.14: What was the type of contribution of the proposed solutions?

The contribution type is classified as follows:

M + T (Method + Tool): A proposal that includes both the methodology and the development of a tool.

M + M/A (Method + Model/Algorithm): A proposal that combines a methodology with the development of a model or algorithm.

MO (Method Only): A proposal that simply proposes a methodology by implementing existing tools or algorithms.

As shown in Fig. 10, most contributions are "method only", which could be because proposing a methodology requires less resources than developing a tool or algorithm, and it is understood that it is more practical to use those that already exist.

4.16. RQ.15: How were the proposed solutions validated?

We classified the studies according to the methodology used to evaluate and test the proposed solutions. This included studies based on:

EV (Experimental Validation): Validation through controlled experiments.

Comp-App (Comparison with other approaches): Comparison with existing methods or solutions.

Comp-Man (Comparison with a manual approach): Comparison with a manually performed validation.

SC (Study Cases): Use of case studies to evaluate the solution.

Fig. 10 shows that most studies reported performing case studies and experimental validation, with the majority achieving a medium level of automation. Additionally, two validation methods were identified that were used only once in the reported studies: a survey using a Likert scale [31] and pilot testing [61].

4.17. RQ.16: How many studies were applied to real projects?

In this study, project types were classified into two categories:

Academic projects: These are conducted solely for academic purposes, meaning they use non-real data and are executed in a controlled environment. The results of these projects do not guarantee the success of the approach in a real industrial context.

Real or industry projects: These are projects that use real industrial data to test the proposed approaches.

As shown in Fig. 11, significant progress was observed in real projects within the automotive domain (12), followed by the entertainment (6), industrial (5), and business (5) domains. In the case of academic projects: (i) the identified domains were entertainment (3), business (5), industrial (2), and automotive (2); and (ii) several of these studies made use of public datasets, such as PURE [43], WebNLG2020 [61], ICON [46], and PixelHelp [67], or created their own datasets based on open-source projects.

In contrast, most real projects opted to build customized datasets tailored to the context in which the study was conducted. Moreover, a considerable proportion of academic studies did not specify the domain in which they were carried out.

5. Discussions

5.1. Bibliometric analysis

This study reviewed 61 primary research articles on test case generation using NLP. Most of these articles were published between 2020 and 2024, reflecting the rapid growth and evolution of AI and NLP in recent years, as well as their integration into software testing practices. Although more than half of the studies were presented at conferences, there was an increase in journal publications in the past year, which could suggest greater maturity and impact of research in the field.

5.2. Proposal components

POS tagging, named entity recognition, and tokenization are the most used techniques in the selected studies. These techniques are fundamental to their application on tasks such as text analysis and extracting relevant information for designing and generating test cases. Since most of the inputs in the proposed approaches are natural language requirements, the use of these techniques is expected.

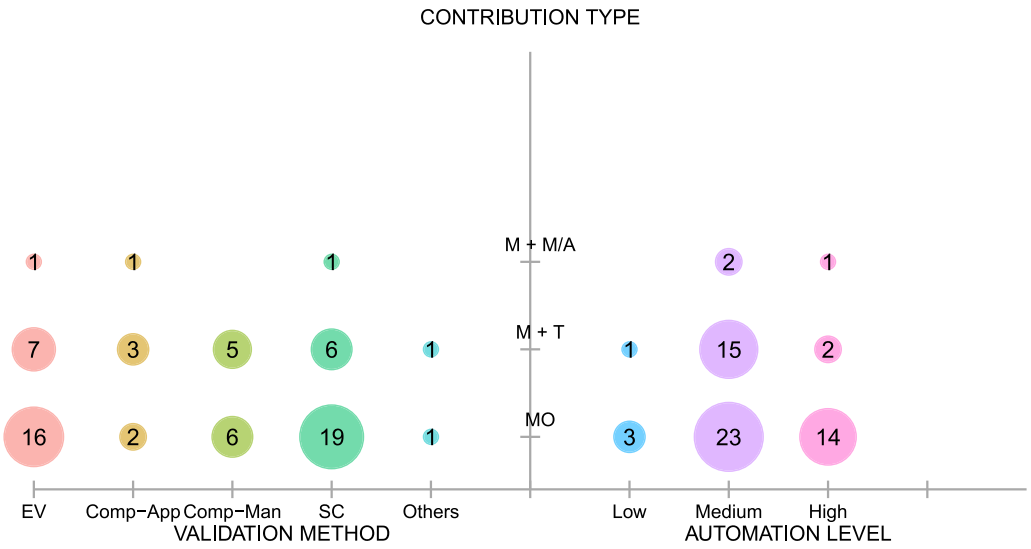


Fig. 10. Validation method, automation level and contribution type.

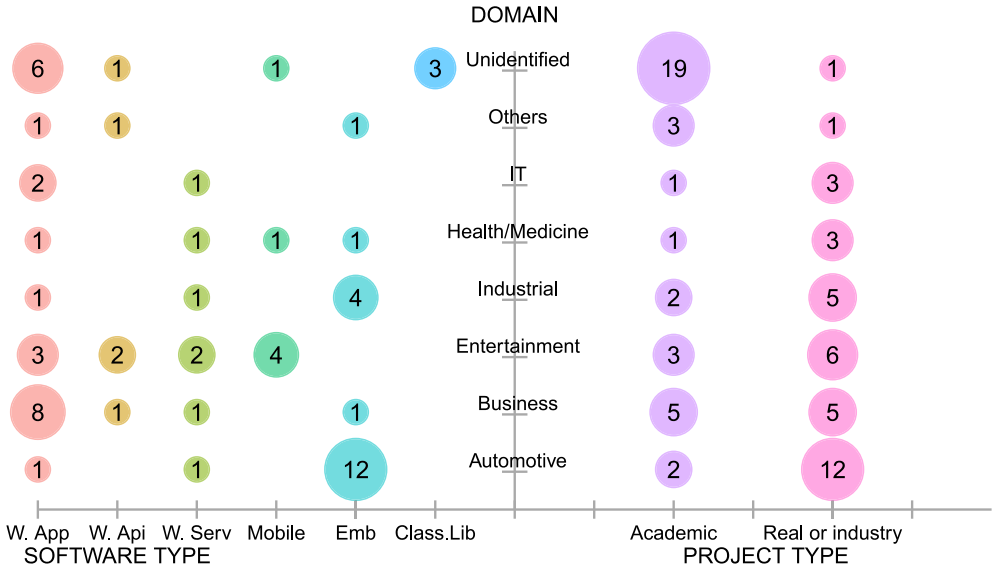


Fig. 11. Software type, software domain, and project type.

The most frequently reported tools in this literature mapping are Stanford Core NLP, NLTK, and spaCy. The reason for this may be that these tools are easy to implement and offer at least a free or trial version. In addition, they have been established in the industry for a long time and have delivered good results in previous studies, which motivates and instills confidence in researchers.

This study identified works reporting the use of LLMs, either to perform specific tasks or to complement approaches based on classical NLP. It was concluded that LLMs are primarily employed for unit test case generation, due to their strong capability to produce code. However, since software testing encompasses a wide variety of test types (beyond unit testing), not all of them require executable code scripts for instance, black box testing or acceptance testing which are generally not addressed by these approaches. Regarding the inputs identified for LLMs, the most common were requirements, source code used for training or fine-tuning, bug reports, and prompts. In contrast, approaches not involving LLMs relied more heavily on natural language requirements. LLMs typically employ pre-training, fine-tuning, and prompt engineering schemes to guide their behavior toward the desired outcomes, since default pre-training may not be sufficient to tackle problems as specific as test case generation in each context. Finally, among the studies using LLMs, most evaluated their proposals in academic projects (9/13). This may be explained by the fact that, in academic environments, implementation, testing, and experimentation are more feasible, whereas in real-world contexts such efforts entail costs that companies are not always willing to bear.

It was observed that most approaches do not integrate test design techniques, even though such techniques maximize the probability of detecting a greater number of faults with minimal effort; this omission limits the comprehensiveness of the proposed solutions. One possible explanation is that the incorporation of test design techniques into automated frameworks requires a prior stage of formalizing and structuring the requirements according to the inputs needed by these techniques (for example, transforming into rules or models the criteria that define partitions or boundary values). This increases the complexity of the approach and may have led researchers to prioritize other aspects of test case generation, at least in the initial or experimental stages of their proposals. Another hypothesis is that research in this area is often driven primarily by the natural language processing community rather than by the software testing community. Consequently, the

focus tends to be on language processing and understanding, while other dimensions of the testing process are given less attention. This represents an opportunity for future work aiming to improve coverage and fault detection through the integration of test design techniques into automated test case generation frameworks.

The most used inputs in the approaches for generating test cases are the requirements, as they contain relevant information about the functionality that needs to be tested. With the help of NLP techniques, this information is extracted to generate the test cases. In the reviewed studies, the majority reported using unstructured requirements, written in natural language, which can lead to many ambiguities, this remains a challenge for researchers in this area.

5.3. Generated test case characteristics

The generated test cases primarily comprise inputs (test values or data) and the expected results, as these two components are often sufficient to evaluate a system's functionalities.

Regarding the ISO 25010 model, 78.7% of the primary studies report an orientation toward some quality characteristic, with functional correctness being the most reported. For studies that do not specify a quality characteristic orientation, it is generally assumed that they implicitly refer to functional correctness, which aligns with the predominant focus on functional testing. Conversely, very few studies assess non-functional quality characteristics-such as performance efficiency, compatibility, interaction capability, reliability, maintainability, flexibility, and protection-even though these aspects have a significant impact on user satisfaction and overall system acceptance [77]. This indicates a gap in current research that future work could address by developing test generation methods that also evaluate non-functional requirements.

5.4. Proposal evaluation

Among performance metrics, precision is the most reported, alongside recall, accuracy, and F1-Score. In this context, precision is calculated as the number of syntactically correct test cases relative to the total number of test cases generated, with most studies reporting values above 80%. The high level of precision may be attributed to the application of NLP in the proposed approaches, which reduces ambiguity in the inputs and improves information extraction compared to more generic approaches, such as the use of LLMs.

Requirements coverage is the most frequently reported type of coverage and the one that reaches the highest levels. This is likely because ensuring requirements coverage is relatively straightforward-it only requires having one test case for each defined requirement. Conversely, few studies reports evaluating the coverage of methods, paths, branches, and code, possibly because the context in which the test cases are generated by the proposed approaches is outside the development environment necessary for assessing these types of coverage. However, among the studies that do evaluate these additional coverage types, the reported levels are generally low (less than 70%), likely because the proposed solutions generate test cases directly from the provided inputs without applying additional techniques or methods, such as test case design techniques, which as mentioned earlier, significantly increase the likelihood of detecting faults when testing each functionality of the system.

5.5. Proposal characteristics

Regarding the type of contribution, it is noteworthy that the proposals categorized as Method + Tool (M+T) and Method + Model/Algorithm (M+M/A) are associated with medium levels of automation. This is possibly because they integrate tailored components (tools or algorithms) that help automate parts of the testing or validation process. Meanwhile, studies categorized as Method Only (MO), which account for most studies (41 out of 61), mostly achieved medium to high automation levels. Although these studies do not develop their own tools or algorithms, they make use of existing ones that have already proven effective in previous research. Most of these "Method Only" contributions were evaluated using experimental validation methods and case studies, facilitating their acceptance and application within the academic community and bridging the gap to industry.

Among the clearly defined domains, Information Technology (IT) and Health/Medicine stand out, followed by Industrial, Entertainment, Business, and Automotive. A significant portion of the works is classified as Not Identified or Others, indicating that many studies do not specify a concrete domain context or cover multiple fields. This reflects the diversity of areas that can benefit from the proposed solutions, as well as the interest in sectors with specific needs.

In terms of software types reported in this mapping, there is a predominance of web applications, web services, and embedded systems. Our findings may indicate that authors are particularly interested in proposing solutions to evaluate web-based systems and embedded systems, likely due to the current demand for these types of systems.

With respect to the type of projects where the proposed approaches are applied, academic projects tend to focus on the Business and Entertainment domains, possibly because the open-source software used in these studies is oriented toward such domains. In contrast, real projects are more frequently observed in the Business and Automotive domains, suggesting the presence of industrial collaborations or the need for more specialized solutions in these sectors. The high number of academic studies without a specified domain may be explained by: (i) the use of public datasets or open-source projects that are not associated with a specific domain; (ii) proof-of-concept works focused on validating feasibility, where the domain is considered irrelevant; (iii) the authors' perception that reporting the domain was not necessary; and (iv) restrictions related to the use of confidential data.

6. Threats to validity

In this section, the threats are analyzed in the context of the four types of validity threats presented in [8]: descriptive validity, theoretical validity, generalization, and interpretive validity.

Descriptive Validity: This refers to the degree to which observations are described accurately and objectively. To mitigate this threat, classifiers were defined for each research question. Based on these, an Excel template was designed to record the information extracted from each article, defining and categorizing the data through an iterative process. This template improved the accuracy of the data extraction process and ensured traceability by allowing consultation whenever necessary. Therefore, this threat is under control.

Theoretical Validity: This refers to the ability to capture what is intended to be captured. This threat arises in two phases of the mapping: the identification and the selection of studies. During the identification phase, relevant studies may have been overlooked. To reduce this threat, a robust search string was designed, consisting of main terms defined through the PI method and alternative terms identified from a set of previously retrieved relevant articles, which were also used to evaluate the search string. Another factor that helps to mitigate this threat is the use of databases with high relevance in software engineering.

During the selection phase, relevant studies may also have been omitted, since the selection was primarily performed by a single author. To address this risk and increase the reliability of the results, a well-defined set of inclusion and exclusion criteria was established. Furthermore, in cases of uncertainty about the classification of a paper, it was included for subsequent, more detailed review. In addition, a second author reviewed each phase of the selection process for both included and excluded articles.

The same procedure was applied to the data extraction process: the first author performed the full reading using Mendeley, highlighting the

Table A.11
List of primary studies.

Id	Title	Year	Project type	Ref.
1	A Natural Language Programming Approach for Requirements-Based Security Testing	2018	Industrial	[52]
2	AC3R: automatically reconstructing car crashes from police reports	2019	Industrial	[31]
3	AI-Driven Testing: Unleashing Autonomous Systems for Superior Software Quality Using Generative AI	2024	Industrial	[58]
4	An approach to mobile application testing based on natural language scripting	2017	Industrial	[78]
5	An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation	2024	Academic	[59]
6	An Ontology-based Approach to Automate the Software Development Process	2018	Industrial	[32]
7	ARTE: Automated Generation of Realistic Test Inputs for Web APIs	2023	Academic	[33]
8	AutoKG - An automotive domain knowledge graph for software testing: A position paper	2021	Academic	[30]
9	Automated Discovery of Valid Test Strings from the Web Using Dynamic Regular Expressions Collation and Natural Language Processing	2012	Academic	[76]
10	Automated Generation of Constraints from Use Case Specifications to Support System Testing	2018	Industrial	[49]
11	Automated Test Case Generation from Input Specification in Natural Language	2022	Academic	[39]
12	Automated Test Case Generation Using T5 and GPT-3	2023	Academic	[61]
13	Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study	2023	Industrial	[65]
14	Automatic Generation of Acceptance Test Cases from Use Case Specifications: An NLP-Based Approach	2022	Industrial	[26]
15	Automatic generation of oracles for exceptional behaviors	2016	Academic	[19]
16	Automatic generation of system test cases from use case specifications	2015	Industrial	[18]
17	Automatic Test Case Generation Method for Large Scale Communication Node Software	2018	Academic	[40]
18	Automatically generating tests from natural language descriptions of software behavior	2013	Academic	[34]
19	Automatically translating bug reports into test cases for mobile apps	2018	Academic	[22]
20	Automating Fault Test Cases Generation and Execution for Automotive Safety Validation via NLP and HIL Simulation	2024	Industrial	[62]
21	Call Me Maybe: Using NLP to Automatically Generate Unit Test Cases Respecting Temporal Constraints	2023	Academic	[46]
22	CAT-LM Training Language Models on Aligned Code and Tests	2023	Academic	[53]
23	CiRA: An Open-Source Python Package for Automated Generation of Test Case Descriptions from Natural Language Requirements	2023	Academic	[63]
24	Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation	2024	Academic	[60]
25	CRAWLABEL: Computing Natural-Language Labels for UI Test Cases	2022	Academic	[35]
26	Documentation-based functional constraint generation for library methods	2021	Academic	[79]
27	Domain Knowledge is All You Need: A Field Deployment of LLM-Powered Test Case Generation in FinTech Domain	2024	Academic	[69]
28	Effective test generation using pre-trained Large Language Models and mutation testing	2024	Academic	[74]
29	Empirical test design strategies using natural language processing	2021	Academic	[28]
30	Enhancing REST API Testing with NLP Techniques	2023	Academic	[41]
31	Generating Abstract Test Cases from User Requirements using MDSE and NLP	2022	Industrial	[36]

(*continued on next page*)

Table A.11 (*continued*).

Id	Title	Year	Project type	Ref.
32	Generating effective test cases for self-driving cars from police reports	2019	Industrial	[20]
33	Generating Executable Test Scenarios from Autonomous Vehicle Disengagements using Natural Language Processing	2024	Academic	[44]
34	Generation of patterns for test cases	2015	Academic	[29]
35	Generation of Test Cases from Software Requirements Using Natural Language Processing	2013	Academic	[70]
36	Integrated and Iterative Requirements Analysis and Test Specification: A Case Study at Kostal	2021	Industrial	[80]
37	Intelligent Generation of Test Cases for a Parallel Testing System: A Case Study on Railway Systems	2024	Academic	[64]
38	Is NLP-based Test Automation Cheaper Than Programmable and Capture&Replay?	2022	Academic	[81]
39	Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction	2023	Academic	[23]
40	Litmus: Generation of test cases from functional requirements in natural language	2012	Industrial	[55]
41	LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities	2023	Industrial	[56]
42	Machine Learning and Constraint Solving for Automated Form Testing	2019	Industrial	[37]
43	Mobile Test Script Generation from Natural Language Descriptions	2023	Academic	[67]
44	Model-Based Testing from Controlled Natural Language Requirements	2014	Industrial	[47]
45	NAT2TESTSCR: Test case generation from natural language requirements based on SCR specifications	2014	Industrial	[24]
46	NLForSpec: Translating natural language descriptions into formal test case specifications	2007	Industrial	[82]
47	NLP-based requirements formalization for automatic test case generation	2021	Industrial	[45]
48	Optimized automated testing: test case generation and maintenance using latent semantic analysis-based TextRank and particle swarm optimization algorithms	2024	Industrial	[75]
49	Research on Test Case Generation Method of Airborne Software Based on NLP	2022	Academic	[57]
50	RETNA: from requirements to testing in a natural way	2004	Academic	[54]
51	SPECMATE: Automated Creation of Test Cases from Acceptance Criteria	2020	Industrial	[25]
52	Supporting Test Case Design on Reasoning Scheme with Natural Language Processing Technique	2021	Academic	[51]
53	Syntactic rules of extracting test cases from software requirements	2016	Academic	[71]
54	Test case generation algorithms and tools for specifications in natural language	2020	Academic	[38]
55	Test case generation and history data analysis during optimization in regression testing: An NLP study	2023	Industrial	[42]
56	Test case generation from natural language requirements based on SCR specifications	2013	Industrial	[27]
57	Test case information extraction from requirements specifications using NLP-based unified boilerplate approach	2024	Academic	[43]
58	Translating code comments to procedure specifications	2018	Academic	[21]
59	UMTG: A toolset to automatically generate system test cases from use case specifications	2015	Industrial	[18]
60	Using deep learning for selenium web UI functional tests: A case-study with e-commerce applications	2023	Academic	[66]
61	Validating, verifying and testing timed data-flow reactive systems in Coq from controlled natural-language requirements	2021	Industrial	[48]

sections from which information was extracted and recording them in a spreadsheet. Subsequently, the second author carried out a detailed review to ensure the quality of the extracted data.

Generalization: This relates to the extent to which the results of the literature mapping can be generalized. This study focuses on synthesizing and systematically structuring the data reported in primary studies related to the generation of test cases using NLP, without delving into the outcomes, so the intent is not to generalize the findings.

Interpretive Validity: Also known as conclusion validity, this is achieved when the conclusions drawn are reasonable based on the extracted data. In this study, the conclusions are not open to interpretation since they are based solely on the results of synthesizing and structuring the data from the primary studies.

7. Conclusions

In this systematic mapping study, a classification and synthesis of studies related to test case generation using NLP were conducted. By applying the proposed methodology, the relevant studies were systematically identified and classified to extract the necessary information to answer the research questions. The results show significant progress in applying NLP to test case generation, with various techniques, tools, algorithms, and methodologies implemented in both academic and real-world projects across multiple industries. It was also observed that few studies incorporate the use of test case design techniques in their proposals, which presents an opportunity for future research in this area. This study provides a comprehensive overview of the use of NLP in test case generation, offering future researchers valuable insights into the NLP-based technologies that hold the greatest potential for test case generation.

CRediT authorship contribution statement

Jordy Navarro: Writing – original draft, Investigation, Conceptualization. **Ronald Ibarra:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. List of the primary studies reviewed in this study

See Table A.11.

Data availability

Data will be made available on request.

References

[1] E.K. Mohd, K. Farmeena, A comparative study of white box, black box and grey box testing techniques, Int. J. Adv. Comput. Sci. Appl. (2012) Available: www.ijacsa.thesai.org.

[2] M. Pawlak, A. Poniszewska-Marańda, Software testing management process for agile approach projects, in: Lecture Notes on Data Engineering and Communications Technologies, vol. 30, 2020, http://dx.doi.org/10.1007/978-3-030-19069-9_3.

[3] IEEE, Guide To the Software Engineering Body of Knowledge SWEBOK, IEEE Computer Society, 2014.

[4] ISTQB, Probador certificado del ISTQB programa de estudio de nivel básico, 2018.

[5] ISO/IEC/IEEE, ISO/IEC/IEEE 29119-2 software and systems engineering-software testing-part 2: Test processes, 2013.

[6] I. Ahsan, W.H. Butt, M.A. Ahmed, M.W. Anwar, A comprehensive investigation of natural language processing techniques and tools to generate automated test cases, in: ACM International Conference Proceeding Series, 2017, http://dx.doi.org/10.1145/3018896.3036375.

[7] H. Ayenew, M. Wagaw, Software test case generation using natural language processing (NLP): A systematic literature review, Artif. Intell. Evol. (2024) http://dx.doi.org/10.37256/aie.5120243220.

[8] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software testing with large language models: Survey, landscape, and vision, 2024, Available: http://arxiv.org/abs/2307.07221.

[9] A. Sezgin, G. Ozkan, E. Cosgun, Leveraging large language models in software testing: A review of applications and challenges, in: ISDFS 2025-13th International Symposium on Digital Forensics and Security, Institute of Electrical and Electronics Engineers Inc., 2025, http://dx.doi.org/10.1109/ISDFS65363.2025.11011986.

[10] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, Inf. Softw. Technol. 64 (2015) http://dx.doi.org/10.1016/j.infsof.2015.03.007.

[11] A. Trudova, M. Dolezel, A. Buchalcevova, Artificial intelligence in software test automation: A systematic literature review, in: ENASE 2020 - Proceedings of the 15th International Conference on Evaluation of Novel Approaches To Software Engineering, 2020, http://dx.doi.org/10.5220/0009417801810192.

[12] M. Islam, F. Khan, S. Alam, M. Hasan, Artificial intelligence in software testing: A systematic review, in: IEEE Region 10 Annual International Conference, Proceedings/TENCON, 2023, pp. 524–529, http://dx.doi.org/10.1109/TENCON58879.2023.10322349.

[13] S. Ajorloo, A. Jamarani, M. Kashfi, M. Haghi Kashani, A. Najafizadeh, A systematic review of machine learning methods in software testing, Appl. Soft Comput. (2024) http://dx.doi.org/10.1016/j.asoc.2024.111805.

[14] B.R. Korraprolu, P. Pinninti, Y.R. Reddy, Test case generation for requirements in natural language - An LLM comparison study, in: ISEC 2025 - Proceedings of the 18th Innovations in Software Engineering Conference, Association for Computing Machinery, Inc, 2025, http://dx.doi.org/10.1145/3717383.3717389.

[15] Q. Zhang, C. Fang, S. Gu, Y. Shang, Z. Chen, L. Xiao, Large language models for unit testing: A systematic literature review, 2025, Available: http://arxiv.org/abs/2506.15227.

[16] V. Garousi, S. Bauer, M. Felderer, NLP-assisted software testing: A systematic mapping of the literature, Inf. Softw. Technol. (2020) http://dx.doi.org/10.1016/j.infsof.2020.106321.

[17] M. Boukhlif, M. Hanine, N. Kharmoum, A.R. Noriega, D.G. Obeso, I. Ashraf, Natural language processing-based software testing: A systematic literature review, IEEE Access 12 (2024) 79383–79400, http://dx.doi.org/10.1109/ACCESS.2024.3407753.

[18] C. Wang, F. Pastore, A. Goknil, L. Briand, Z. Iqbal, Automatic generation of system test cases from use case specifications, in: 2015 International Symposium on Software Testing and Analysis, ISSTA 2015 - Proceedings, 2015, pp. 385–396, http://dx.doi.org/10.1145/2771783.2771812.

[19] A. Goffi, A. Gorla, M.D. Ernst, M. Pezzè, Automatic generation of oracles for exceptional behaviors, in: ISSTA 2016 - Proceedings of the 25th International Symposium on Software Testing and Analysis, 2016, pp. 213–224, http://dx.doi.org/10.1145/2931037.2931061.

[20] A. Gambi, T. Huynh, G. Fraser, Generating effective test cases for self-driving cars from police reports, in: ESEC/FSE 2019 - Proceedings of the 2019 27th ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 257–267, http://dx.doi.org/10.1145/3338906.3338942.

[21] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M.D. Ernst, M. Pezze, S.D. Castellanos, Translating code comments to procedure specifications, in: ISSTA 2018 - Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2018, http://dx.doi.org/10.1145/3213846.3213872.

[22] M. Fazzini, M. Prammer, M. D'Amorim, A. Orso, Automatically translating bug reports into test cases for mobile apps, in: ISSTA 2018 - Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2018, pp. 141–152, http://dx.doi.org/10.1145/3213846.3213869.

[23] S. Kang, J. Yoon, S. Yoo, Large language models are few-shot testers: Exploring LLM-based general bug reproduction, 2023, Available: https://issues.apache.org/jira/browse/MATH-370.

[24] G. Carvalho, F. Barros, F. Lapschies, U. Schulze, J. Peleska, NAT2TESTSCR: Test case generation from natural language requirements based on SCR specifications, Sci. Comput. Program. 95 (P3) (2014) 275–297, http://dx.doi.org/10.1016/j.scico.2014.06.007.

[25] J. Fischbach, A. Vogelsang, D. Spies, A. Wehrle, M. Junker, D. Freudenstein, SPECMATE: Automated creation of test cases from acceptance criteria, in: Proceedings - 2020 IEEE 13th International Conference on Software Testing, Verification and Validation, ICST 2020, 2020, pp. 321–331, http://dx.doi.org/10.1109/ICST46399.2020.00040.

[26] C. Wang, F. Pastore, A. Goknil, L.C. Briand, Automatic generation of acceptance test cases from use case specifications: An NLP-based approach, IEEE Trans. Softw. Eng. 48 (2) (2022) 585–616, http://dx.doi.org/10.1109/TSE.2020.2998503.

[27] G. Carvalho, F. Barros, F. Lapschies, U. Schulze, J. Peleska, Test case generation from natural language requirements based on SCR specifications, 2013.

[28] T.S. Abishek, A. Viswanathan, A.K. Pujari, V.S. Felix Enigo, Empirical test design strategies using natural language processing, in: Lecture Notes in Networks and Systems, 2021, pp. 203–217, http://dx.doi.org/10.1007/978-981-15-7345-3_17.

[29] V. Vani, G.S. Mahalakshmi, J. Betina Antony, Generation of patterns for test cases, Indian J. Sci. Technol. 8 (24) (2015) http://dx.doi.org/10.17485/ijst/2015/v8i24/80176.

[30] V. Kesri, A. Nayak, K. Ponnalagu, AutoKG - An automotive domain knowledge graph for software testing: A position paper, in: Proceedings - 2021 IEEE 14th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2021, 2021, pp. 234–238, http://dx.doi.org/10.1109/ICSTW52544.2021.00047.

[31] T. Huynh, A. Gambi, G. Fraser, AC3R: Automatically reconstructing car crashes from police reports, 2019, Available: https://github.com/SoftLegend/AC3R-Demo.

[32] A. Kathirgamasegaran, R. Selvaratnam, S. Srijeevahan, G. Nahanaa, An ontology-based approach to automate the software development process, 2018.

[33] J.C. Alonso, A. Martin-Lopez, S. Segura, M. García, A. Ruiz-Cortés, ARTE: Automated generation of realistic test inputs for web APIs, 2023, Available: http://dbpedia.org/resource/George_R._R.

[34] S. Kamalakar, S.H. Edwards, T.M. Dao, Automatically generating tests from natural language descriptions of software behavior, in: ENASE 2013 - Proceedings of the 8th International Conference on Evaluation of Novel Approaches To Software Engineering, 2013, pp. 238–245, http://dx.doi.org/10.5220/0004566002380245.

[35] Y. Liu, R. Yandrapally, A.K. Kalia, S. Sinha, R. Tzoref-Brill, A. Mesbah, CRAWLABEL: Computing natural-language labels for UI test cases, in: Proceedings - 3rd ACM/IEEE International Conference on Automation of Software Test, AST 2022, 2022, pp. 103–114, http://dx.doi.org/10.1145/3524481.3527229.

[36] S.C. Allala, J.P. Sotomayor, D. Santiago, T.M. King, P.J. Clarke, Generating abstract test cases from user requirements using MDSE and NLP, in: IEEE International Conference on Software Quality, Reliability and Security, QRS, 2022, pp. 744–753, http://dx.doi.org/10.1109/QRS57517.2022.00080.

[37] D. Santiago, J. Phillips, P. Alt, B. Muras, T.M. King, P.J. Clarke, Machine learning and constraint solving for automated form testing, 2019.

[38] Y. Aoyama, T. Kuroiwa, N. Kushiro, Test Case Generation Algorithms and Tools for Specifications in Natural Language, IEEE, 2020.

[39] T. Li, X. Lu, H. Xu, Automated test case generation from input specification in natural language, in: Proceedings - 2022 IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW 2022, 2022, pp. 258–261, http://dx.doi.org/10.1109/ISSREW55968.2022.00076.

[40] K. Kikuma, T. Yamada, K. Ueda, A. Fukuda, Automatic test case generation method for large scale communication node software, in: Lecture Notes on Data Engineering and Communications Technologies, vol. 17, 2018, pp. 492–503, http://dx.doi.org/10.1007/978-3-319-75928-9_44.

[41] M. Kim, et al., Enhancing REST API testing with NLP techniques, in: ISSTA 2023 - Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 1232–1243, http://dx.doi.org/10.1145/3597926.3598131.

[42] A. Gupta, R.P. Mahapatra, Test case generation and history data analysis during optimization in regression testing: An NLP study, Cogent Eng. 10 (2) (2023) http://dx.doi.org/10.1080/23311916.2023.2276495.

[43] J.W. Lim, et al., Test case information extraction from requirements specifications using NLP-based unified boilerplate approach, J. Syst. Softw. 211 (2024) http://dx.doi.org/10.1016/j.jss.2024.112005.

[44] Q. Song, R. Anderberg, H. Olsson, P. Runeson, Generating executable test scenarios from autonomous vehicle disengagements using natural language processing, in: Proceedings - 2024 IEEE/ACM 19th Symposium on Software Engineering for Adaptive and Self-Managing Systems, SEAMS 2024, 2024, pp. 98–104, http://dx.doi.org/10.1145/3643915.3644098.

[45] R. Gröpler, V. Sudhi, E. José, C. García, A. Bergmann, NLP-based requirements formalization for automatic test case generation, 2021, Available: http://ceur-ws.org.

[46] A. Blasi, A. Gorla, M.D. Ernst, M. Pezzè, Call me maybe: Using NLP to automatically generate unit test cases respecting temporal constraints, in: ACM International Conference Proceeding Series, 2022, http://dx.doi.org/10.1145/3551349.3556961.

[47] G. Carvalho, F. Barros, F. Lapschies, U. Schulze, J. Peleska, Model-Based Testing from Controlled Natural Language Requirements, Springer Verlag, 2014, http://dx.doi.org/10.1007/978-3-319-05416-2.

[48] G. Carvalho, I. Meira, Validating, verifying and testing timed data-flow reactive systems in Coq from controlled natural-language requirements, Sci. Comput. Program. 201 (2021) http://dx.doi.org/10.1016/j.scico.2020.102537.

[49] C. Wang, F. Pastore, L. Briand, Automated generation of constraints from use case specifications to support system testing, in: Proceedings - 2018 IEEE 11th International Conference on Software Testing, Verification and Validation, ICST 2018, 2018, pp. 23–33, http://dx.doi.org/10.1109/ICST.2018.00013.

[50] C. Wang, F. Pastore, A. Goknil, L.C. Briand, Z. Iqbal, UMTG: A toolset to automatically generate system test cases from use case specifications, in: 2015 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ESEC/FSE 2015 - Proceedings, 2015, pp. 942–945, http://dx.doi.org/10.1145/2786805.2803187.

[51] N. Kushiro, Y. Ogata, Supporting test case design on reasoning scheme with natural language processing technique, in: Proceedings - 2021 IEEE International Conference on Big Data, Big Data 2021, 2021, http://dx.doi.org/10.1109/BigData52589.2021.9671718.

[52] P.X. Mai, F. Pastore, A. Goknil, L.C. Briand, A natural language programming approach for requirements-based security testing, in: Proceedings - International Symposium on Software Reliability Engineering, ISSRE, 2018, pp. 58–69, http://dx.doi.org/10.1109/ISSRE.2018.00017.

[53] N. Rao, K. Jain, U. Alon, C. Le Goues, V.J. Hellendoorn, CAT-LM training language models on aligned code and tests, 2023, Available: https://github.com/RaoNikitha/CAT-LM.

[54] R. Boddu, L. Guo, S. Mukhopadhyay, B. Cukic, RETNA: From requirements to testing in a natural way, in: Proceedings of the IEEE International Conference on Requirements Engineering, 2004, pp. 262–271, http://dx.doi.org/10.1109/ICRE.2004.1335683.

[55] A. Dwarakanath, S. Sengupta, Litmus: Generation of test cases from functional requirements in natural language, 2012.

[56] S. Yu, C. Fang, Y. Ling, C. Wu, Z. Chen, LLM for test script generation and migration: Challenges, capabilities, and opportunities, 2023, Available: http://selendroid.io.

[57] C. Chao, Q. Yang, X. Tu, Research on test case generation method of airborne software based on NLP, 2022.

[58] M. Helmy, O. Sobhy, F. Elhusseiny, AI-driven testing: Unleashing autonomous systems for superior software quality using generative AI, in: 2024 International Telecommunications Conference, ITC-Egypt 2024, 2024, pp. 202–207, http://dx.doi.org/10.1109/ITC-Egypt61547.2024.10620598.

[59] M. Schäfer, S. Schäfer, S. Nadi, A. Eghbali, F. Tip, An empirical evaluation of using large language models for automated unit test generation, 2023, http://dx.doi.org/10.6084/m9.figshare.23653371.

[60] S. Karpurapu, et al., Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation, IEEE Access 12 (2024) 8715–58721, http://dx.doi.org/10.1109/ACCESS.2024.3391815.

[61] A. Mathur, S. Pradhan, P. Soni, D. Patel, R. Regunathan, Automated test case generation using T5 and GPT-3, in: 2023 9th International Conference on Advanced Computing and Communication Systems, ICACCS 2023, 2023, pp. 1986–1992, http://dx.doi.org/10.1109/ICACCS57279.2023.10112971.

[62] A. Amyan, M. Abboush, C. Knieke, A. Rausch, Automating fault test cases generation and execution for automotive safety validation via NLP and HIL simulation, Sensors 24 (10) (2024) http://dx.doi.org/10.3390/s24103145.

[63] J. Frattini, J. Fischbach, A. Bauer, CiRA: An open-source python package for automated generation of test case descriptions from natural language requirements, in: Proceedings - 31st IEEE International Requirements Engineering Conference Workshops, 2023, pp. 68–71, http://dx.doi.org/10.1109/REW57809.2023.00019.

[64] Y. Jiao, J. Wang, R. Li, F.-Y. Wang, H. Zhao, G. Xiong, Intelligent generation of test cases for a parallel testing system: A case study on railway systems, IEEE Intell. Transp. Syst. Mag. (2024) 2–13, http://dx.doi.org/10.1109/mits.2024.3435828.

[65] J. Fischbach, et al., Automatic creation of acceptance tests by extracting conditionals from requirements: NLP approach and case study, 2023, Available: www.cira.bth.se/demo.

[66] Z. Khaliq, D.A. Khan, S.U. Farooq, Using deep learning for selenium web UI functional tests: A case-study with e-commerce applications, Eng. Appl. Artif. Intell. 117 (2023) http://dx.doi.org/10.1016/j.engappai.2022.105446.

[67] C. Li, Y. Xiong, Z. Li, W. Yang, M. Pan, Mobile test script generation from natural language descriptions, in: IEEE International Conference on Software Quality, Reliability and Security, QRS, 2023, pp. 349–359, http://dx.doi.org/10.1109/QRS60937.2023.00042.

[68] F. Toledo, Introducción a Las Pruebas De Sistemas De información, 2024.

[69] Z. Xue, et al., Domain knowledge is all you need: A field deployment of LLM-powered test case generation in FinTech domain, in: Proceedings - International Conference on Software Engineering, 2024, pp. 314–315, http://dx.doi.org/10.1145/3639478.3643087.

[70] R.P. Verma, M.R. Beg, Generation of test cases from software requirements using natural language processing, in: International Conference on Emerging Trends in Engineering and Technology, ICETET, 2013, pp. 140–147, http://dx.doi.org/10.1109/ICETET.2013.45.

[71] S. Masuda, T. Matsuodani, K. Tsuda, Syntactic rules of extracting test cases from software requirements, in: ACM International Conference Proceeding Series, 2016, pp. 12–17, http://dx.doi.org/10.1145/3012258.3012262.

[72] K. Naik, P. Tripathy, Software testing and quality assurance: Theory and practice, 2008.

[73] R. Grinwald, E. Harel, M. Hael Orgad, S. Uel Ur, A. Ziv, User defined coverage-a tool supported methodology for design verification, 1998.

[74] A.M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, M.C. Desmarais, Effective test generation using pre-trained large language models and mutation testing, 2024.

[75] B. Swathi, S.S. Kolisetty, Optimized automated testing: test case generation and maintenance using latent semantic analysis-based TextRank and particle swarm optimization algorithms, Int. J. Electr. Comput. Eng. 14 (2024) http://dx.doi.org/10.11591/ijece.v14i4.pp4315-4324.

[76] M. Shahbaz, P. McMinn, M. Stevenson, Automated discovery of valid test strings from the web using dynamic regular expressions collation and natural language processing, Proc. - Int. Conf. Qual. Softw. (2012) http://dx.doi.org/10.1109/QSIC.2012.15.

[77] S. Andreas, L. Tilo, Software testing foundations: A study guide for the certified tester exam, 2021.

[78] C. Tao, J. Gao, T. Wang, An approach to mobile application testing based on natural language scripting, in: Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE, 2017, pp. 260–265, http://dx.doi.org/10.18293/SEKE2017-170.

[79] R. Jiang, Z. Chen, Y. Pei, M. Pan, T. Zhang, X. Li, Documentation-based functional constraint generation for library methods, Softw. Test. Verif. Reliab. 31 (8) (2021) http://dx.doi.org/10.1002/stvr.1785.

[80] C. Wiecher, J. Fischbach, J. Greenyer, A. Vogelsang, C. Wolff, R. Dumitrescu, Integrated and iterative requirements analysis and test specification: A case study at kostal, 2021.

[81] M. Leotta, F. Ricca, S. Stoppa, A. Marchetto, Is NLP-based test automation cheaper than programmable and capture & replay?, 2022, Available: https://testrigor.com.

[82] D. Leitão, D. Torres, F. Barros, NLForSpec: Translating natural language descriptions into formal test case specifications, 2007.